

COMPILER DESIGN

ASSIGNMENT

Name : SUDHARSAN D

Register Number : 192524063

Course Code : CSA1405

Course Name : Compiler Design

Slot : C

Institution : SIMATS ENGINEERING

Faculty Name : Dr. S. Sivagami

Q1:

a.) Error Identification:-

- * `int 1num = 10;` - Invalid variable name.
- * Missing ; after `float b = 5.5.`
- * `add(10)` - Missing one argument.
- * `if (c = 20)` - Use `==` instead of `=`.
- * Missing ; after `printf()`.
- * `d` not declared.
- * `arr[5]` - Array out of bounds.
- * `*p = 100;` - Uninitialized pointer.
- * `x/y` - Division by zero.
- * `while (x < 15)` - Infinite loop.

b.) Classification:-

- * Lexical : `1num`
- * Syntax : Missing semicolons.
- * Semantic : wrong function argument; undeclared
- * Logical : `if (c = 20)`, infinite loop.
- * Runtime : Array out of bounds, pointer error, divide by zero.

c.) Compilation Behavior:-

- Compile - time : Invalid variable, missing ;, wrong function call, undeclared variable.
- Runtime : Array error, pointer error, division by zero, infinite loop.

d.) Code correction:-

- ✓ Change hum $\rightarrow$ num
- ✓ Add missing ;
- ✓ Use add (10, 10)
- ✓ Change $= \rightarrow ==$
- ✓ Declare ind d;
- ✓ Use arr [4]
- ✓ Initialize pointer.
- ✓ Set y = 2.
- ✓ Add $x++$ inside loop.

(e.) Impact Analysis.

- * Pointer error $\rightarrow$ program crash.
- * Division by zero $\rightarrow$ Runtime error.
- * Infinite loop $\rightarrow$ Program never stops.

f.) Compiler Improvements:-

- * Better error messages with line numbers.
- * Static analysis to detect runtime errors early.

Q2.

a.) Error Identification:-

Line	Invalid Token.
3	2 value.
4	rate @
5	'ab'
6.	10 \$5
8	num#1
9	"Hells world.
10	09.
11	0xG1
15	% count.
16	12.3.4, "Test

b.) Error Classification:-

⇒ Identifier Rules: 2 value, rate @, num#1, % count.

⇒ Constant Rules: 'ab', 09, 0xG1, 12.3.4

⇒ String / Character Rules: "Hells world", "Test", 'ab'

⇒ operator / Symbol Rules: 10 \$5.

c.) Token Correction:-

* 2 value → value2.

* rate @ → rate.

* 'ab' → 'a'

* 10 \$5 → 10 + 5.

- * num # 1 $\rightarrow$ num 1
- * "Hello world" $\rightarrow$ "Hello World".
- * 09 $\rightarrow$ 9.
- * 0xG, $\rightarrow$ 0xA.
- * %.count $\rightarrow$.count.
- * 12.3.4 $\rightarrow$ 12.34
- * "Test" $\rightarrow$ "Test"

d.) Compiler Behavior:-

- * The lexical analyzer reads the program character by character and converts it into tokens.
- * It reports lexical errors with line numbers.
- * It usually continues scanning using error recovery instead of stopping immediately.

e.) Advanced Thinking:-

$\Rightarrow$ Rule: An identifier must begin with a letter or, not a digit.

$\Rightarrow$ Better Error Messages: Show the line number, highlight the invalid token, and suggest the correct form.

* (eg:- 2 value $\rightarrow$ Value 2).

Q3.

a.) Identify Errors:-

Line

Error.

6

@num - Invalid identifier.

7. Missing : after
float x = 10.0
9. multiply (2) - missing one argument
10. missing) in if (y == 10)
15. missing) in if (y == 15)
17. arr [3] - Array out of bounds.
19. * P = 50 - uninitialized pointer.
21. + 0 - Redundant expression
- 22-24. Infinite loop (a not incremented).

b.) Classification:-

- * Lexical : @ num
- * Syntax : missing ; , missing)
- * Semantic : multiply (2)
- * Intermediate code : Expression handling / operate.
precedence.
- * Optimization : + 0
- * Runtime : Division by zero, array out of bounds,
invalid pointer.

c.) Compilation VS Execution:-

- Compile - time : Invalid identifier, missing ; , missing)
wrong function call.
- Runtime : Division by zero, Pointer error, array
error, infinite loop.
- Early compile errors. Stop compilation, so later
errors may not be detected.

d.) Program Correction:-

- * @num $\rightarrow$ num
- * Add missing;
- * Multiply(2, 5)
- * if (y == 10)
- * Set b = 2.
- * Use arr [2]
- * Initialize pointer.
- * Remove + 0
- * Add a++ inside while.

e.) Deep Analysis:-

- $\Rightarrow$ Pointer error: May cause Program Crash.
- $\Rightarrow$ Array bounds: Can access invalid memory.
- $\Rightarrow$ Operator precedence: Wrong intermediate code may produce incorrect result.

f.) Advanced Task:-

- * Use static analysis to detect runtime errors early.
- * Provide clear error messages with suggestions.
- * Optimization removes redundant expressions like + 0 to improve efficiency.